



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	Swetha Nagasri
Student Name	Myadagoni Ramya Sree
Roll Number	2520030094
Branch	CSE
Course Outcome	CO-3

Case Study Topic : Razorpay Backend Dependency Analysis using DFS and Topological Sort

Work Done Summary :

In this case study, we developed a Java program to analyze software module dependencies using Depth First Search (DFS), Topological Sorting, and Cycle Detection techniques. The dependency graph was represented using adjacency lists, where each module was treated as a graph vertex and dependencies were represented as directed edges.

Using Java collections such as HashMap and ArrayList, we implemented DFS traversal with WHITE, GREY, and BLACK color marking. The algorithm successfully detected cyclic dependencies by identifying back edges during traversal. This approach helps software engineers identify dependency issues and generate valid build orders when no cycles exist.

Implementation Details:

1. Represented software modules and dependencies using an adjacency list graph structure.
2. Implemented DFS traversal using Java HashMap and ArrayList collections.
3. Used WHITE, GREY, and BLACK color marking for cycle detection.
4. Identified back edges during DFS traversal to detect cycles.
5. Generated topological ordering for acyclic dependency graphs.
6. Reported cycle information to help developers debug build failures quickly.

Result: Output Screenshots:

```
Modules Loaded Successfully

Starting DFS Traversal...

Visiting: auth
Visiting: ledger
Visiting: fraud
Visiting: notify
Visiting: gateway
Visiting: admin-ui
Visiting: payments

Back Edge Found: fraud -> ledger

Cycle Detected!

Cycle Path:
ledger -> fraud -> ledger

Dependency Analysis Completed
```

The DFS-based Topological Sort and Cycle Detection algorithm was used to analyze module dependencies in the Razorpay backend build system.

Modules considered in the dependency graph :

auth, payments, kyc, ledger, fraud, notify, admin-ui, customer-ui, gateway

Dependency relationships :

- auth → ledger
- payments → auth
- payments → fraud
- kyc → auth
- ledger → fraud
- fraud → notify
- fraud → ledger
- admin-ui → payments
- admin-ui → kyc
- customer-ui → payments
- gateway → admin-ui
- gateway → customer-ui
- notify → gateway

During DFS traversal, a back edge was detected :

fraud → ledger

Cycle identified :

ledger → fraud → ledger

Result :

Dependency Cycle Detected

Time Complexity:

- DFS Traversal: $O(V + E)$
- Cycle Detection: $O(V + E)$
- Topological Sort: $O(V + E)$
- Graph Construction: $O(V + E)$

Where :

- V = Number of Vertices (Modules)
- E = Number of Edges (Dependencies)

Performance Analysis :

- Number of Modules = 9
- Number of Dependencies = 13
- DFS visits each vertex only once
- DFS explores each edge only once
- Cycle detection occurs during traversal without additional scans
- Suitable for large software dependency management systems

Conclusion :

The DFS-based cycle detection algorithm successfully identified cyclic dependencies in the build graph. By detecting the cycle between ledger and fraud modules, the system can prevent build failures and assist developers in resolving dependency issues before deployment.

GitHub Link:

Student Signature	Faculty Signature